

Choosing AI and Statistical Methods for LCIP

Executive summary

The strongest methodological choice for an LCIP-style MVP is **not** “AI everywhere”. It is **statistical methods for the core signal layer** and **AI for the language and retrieval layer**. For a London crime intelligence demonstrator, the highest-value, lowest-regret path is to use transparent statistical methods for trending, forecasting, and anomaly detection; use simple tabular ML only where it clearly beats baselines; and reserve LLM-based AI for grounded retrieval, analyst-facing explanation, and summarisation with citations. That split best matches LCIP’s own positioning as a local-first crime intelligence platform with trends, forecasting, anomaly detection, and an AI investigative assistant, while also aligning with NIST’s emphasis on validity, reliability, accountability, transparency, explainability, and risk-balancing for trustworthy AI. ¹

For forecasting specifically, the evidence base still favours strong statistical baselines unless you have **many related series, rich external covariates**, and a clear need for nonlinear pooling across them. The M4 competition found that combinations dominated and that machine-learning methods did not simply sweep away classical methods, while later competition evidence shows ML doing better when large panels and external features are available. DeepAR is expressly designed for large collections of related series, and TFT is designed for multi-horizon problems with mixed covariates; that is a different regime from “a modest number of borough-by-offence monthly series plus a dashboard”. ²

For policing-adjacent use cases, governance is not optional. UK ICO guidance explicitly foregrounds accountability, transparency, statistical accuracy, and fairness in AI systems, while the EU AI Act’s Annex III places multiple law-enforcement AI use cases into the high-risk category and couples that with requirements around risk management, data governance, technical documentation, logging, transparency, human oversight, and accuracy/robustness. Even if LCIP is only a demonstrator, that regulatory posture should influence the architecture: avoid person-level risk scoring and keep the MVP focused on descriptive and analyst-support functions rather than automated operational decisions about individuals. ³

The practical recommendation for the LCIP MVP is therefore straightforward. Use **SQL/pandas + BI-style analytics** for descriptive dashboards; **seasonal naïve + ETS/SARIMAX/Prophet** for forecasting; **STL residual analysis plus CUSUM/EWMA** for anomaly detection; **logistic regression or XGBoost** only for bounded, interpretable tabular classification tasks; **hybrid BM25 + embeddings + metadata filters** for retrieval; and **template-first plus grounded LLM summarisation** for the analyst assistant. Wrap the whole thing with **rolling-origin backtesting, MLflow lineage/versioning, OpenTelemetry traces/logs/metrics**, and **model/data documentation**. That gives you a polished AI-engineering story without overclaiming or overfitting the problem. ⁴

LCIP context and the decision framework

LCIP is best understood as a **decision-support and analysis platform**, not as a blank cheque for sophisticated modelling. Its repository and README frame it around London crime data, trend analysis, forecasting, anomaly detection, and a local AI assistant; the stack is local-first and includes local models via Ollama plus pgvector-backed retrieval. That matters, because methodology should follow intended

use, not hype. In Google's engineering guidance, the first move is to keep the pipeline solid, instrument metrics early, and keep the first model simple; complexity is justified only after simple baselines plateau. ⁵

A robust choice framework for LCIP has nine criteria.

Data volume and structure. If you have short or medium-length univariate or lightly multivariate series, sparse labels, and a need to explain movement in each series, statistical methods usually dominate on practicality. If you have hundreds or thousands of related series, dense event histories, many exogenous drivers, and clear evidence of nonlinear interactions, global ML or deep time-series models become more attractive. DeepAR's own formulation is built around "a large number of related time series", and TFT targets multi-horizon problems with static, known-future, and exogenous covariates. ⁶

Problem type. Forecasting, anomaly detection, classification, retrieval, and summarisation are not the same class of problem and should not be forced into one methodology. NIST explicitly frames AI appropriateness as contextual and tied to trustworthiness characteristics, risks, impacts, costs, and benefits. LCIP should therefore separate its **signal-generation layer** from its **language interface layer**. ⁷

Latency and serving pattern. If the output must be fast, repeatable, and easily batchable, statistical models and lightweight tabular models are ideal. BM25 retrieval is also extremely fast and lightweight. Haystack's documentation notes that BM25 is often used for exact matches such as names, IDs, and well-defined patterns, and that it is "very lightweight and simple", yet can be hard to beat on out-of-domain data. That profile fits internal search over crime categories, borough names, incident IDs, and policy documents. ⁸

Interpretability and domain validity. NIST's trustworthy AI guidance explicitly calls out validity/reliability, accountability/transparency, and explainability/interpretability as core characteristics, and warns that trade-offs need contextual justification. In a policing analysis context, that means a method should support analyst sense-making, hypothesis testing, and defensible narratives, not just high leaderboard scores. College of Policing and related professional guidance place clear value on hypothesis-led analytical products and explicit reasoning. ⁹

Regulatory and rights risk. The closer a system gets to evaluating individuals, prioritising interventions, or influencing investigative action, the more heavily you should bias toward simple, auditable methods and human review. Annex III of the EU AI Act lists multiple law-enforcement uses as high-risk, including systems intended to assess victimisation risk, evaluate evidence reliability, or assess re-offending risk, and the same chapter foregrounds requirements for technical documentation, logging, human oversight, transparency, data governance, and accuracy. Even outside the EU legal perimeter, that is a useful benchmark for responsible architecture. ¹⁰

Cost, ops, and maintenance. Google's Rules of ML and the "Hidden Technical Debt" literature both make the same broad point: complexity that is not buying you disproportionate value becomes an operational liability. Deep models, LLMs, and multi-stage retrieval-generation stacks carry more dependencies, more failure modes, more monitoring surfaces, and harder reproducibility than ETS, SARIMAX, or logistic regression. ¹¹

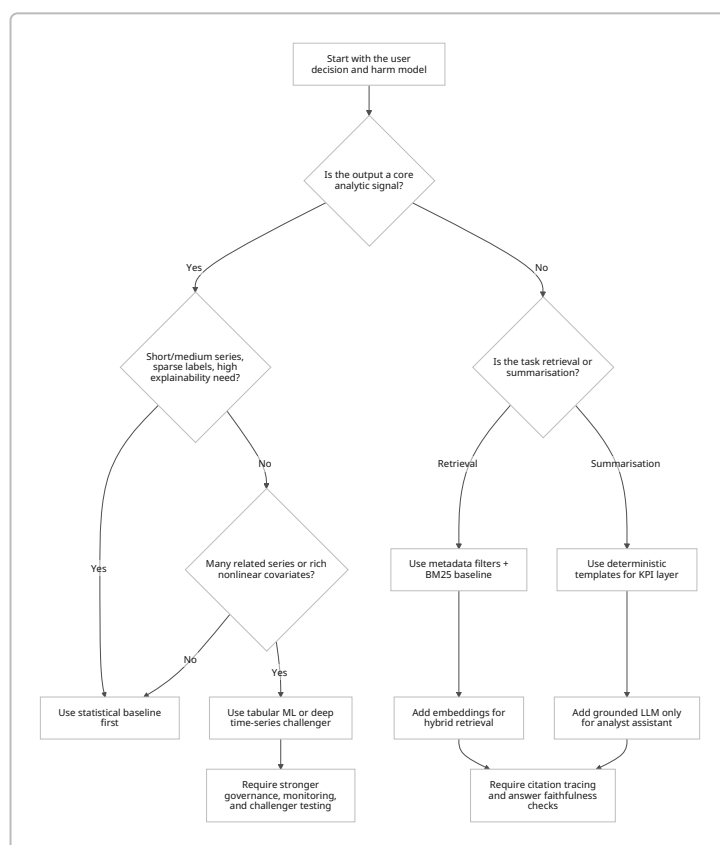
Explainability and reproducibility. ICO guidance is unusually relevant here because it explicitly couples AI governance with transparency, fairness, and statistical accuracy. For LCIP, reproducibility

means fixed data snapshots, versioned preprocessing, pinned model artefacts, stable prompts, and observable serving traces. Explainability means you can answer: what data, what window, what transformation, what model version, what evidence, and why this answer. ¹²

Auditability. MLflow’s model registry is valuable not because it is fashionable but because it gives lineage, versioning, aliasing, tagging, and traceability from experiment to deployment. OpenTelemetry complements that by standardising traces, metrics, and logs across the application path. For an LCIP demonstrator, those capabilities let you show mature operational discipline in interviews. ¹³

Failure mode severity. If an error merely creates a slightly clumsy narrative in an analyst assistant, an LLM may be acceptable with citations and abstention. If an error creates a misleading risk signal or pushes the wrong geographic priority, you should strongly prefer deterministic or statistically interpretable methods. NIST is explicit that high-consequence contexts require proportionate transparency, monitoring, human intervention, and harm minimisation. ⁷

The decision logic for LCIP can be summarised like this:



That flow is a synthesis of the NIST trustworthiness model, Google’s ML engineering rules, RAG literature, and competition evidence on forecasting regimes. ¹⁴

Method map by LCIP use case

The comparison below is the short answer for an interview panel.

Use case	Recommended approach for LCIP MVP	Why	Main trade-offs	Example tools
Forecasting borough/offence counts	Seasonal naïve baseline + ETS/SARIMAX/Prophet ; only add global ML if you later pool many related series with rich exogenous covariates. ¹⁵	Strong on interpretability, fast retraining, good for modest series, easy backtesting and intervals.	Can underfit complex nonlinear interactions; ML may outperform if data regime changes.	statsmodels, prophet, sktime, pmdarima
Anomaly detection on crime counts or shifts	STL + residual thresholds + CUSUM/EWMA first; Isolation Forest as challenger for multivariate anomalies. ¹⁶	Transparent and auditable for shift/change detection; Isolation Forest adds multivariate flexibility without labels.	More opaque once you move to tree-ensemble anomaly scores; threshold tuning still needs judgement.	statsmodels, scipy, scikit-learn, pyod
Classification of alert severity / routing / analyst intent	Logistic regression for first production model; XGBoost if nonlinear interactions are clearly useful. ¹⁷	Well-calibrated, explainable probabilities for simple models; XGBoost is state-of-the-art on many tabular tasks.	XGBoost needs additional explainability/calibration work; avoid person-level risk scoring in MVP.	scikit-learn, xgboost, shap
Retrieval over documents, notes, policies, and methodology	Hybrid BM25 + embeddings + metadata filters ; generation only after retrieval. ¹⁸	BM25 is excellent for exact-match and out-of-domain terms; embeddings add semantic recall; RAG adds provenance.	More moving parts than plain search; answer quality now depends on both retrieval and generation.	PostgreSQL/pgvector, Elasticsearch/OpenSearch, sentence-transformers, Haystack or LlamaIndex
Summarisation and analyst assistant	Template-first for dashboards; grounded local LLM summarisation only for assistant workflows with citations and abstention. ¹⁹	Maximises reproducibility and audibility in the KPI layer, while still showcasing AI in the assistant layer.	Free-form LLM summaries can hallucinate or omit critical facts; must be constrained.	Ollama, local LLMs, prompt templates, citation checks

The table is the operational conclusion of the research: **classical/statistical where the platform produces evidence, AI where the platform interprets evidence for humans.** ²⁰

Forecasting

For LCIP forecasting, the correct baseline stack is:

1. **Seasonal naïve** as the irreducible baseline.
2. **ETS / Exponential Smoothing** when the signal is essentially level-trend-seasonality.
3. **SARIMAX** when exogenous regressors matter.
4. **Prophet** when you want decomposable trend/seasonality/holiday effects with analyst-in-the-loop tuning.
5. **Global ML/deep models** only if you later build a wide panel of related series and covariates.

That ordering is not conservative for its own sake. Prophet's original paper explicitly emphasises a decomposable model with interpretable parameters and analyst-in-the-loop adjustment. ETS and SARIMAX sit comfortably in the same explainable family. Meanwhile, M4 warns against assuming ML dominance, and DeepAR/TFT define the data regime in which more sophisticated models become justified. ²¹

A minimal statistical forecast sketch:

```
# Statistical first-pass
y = monthly_count_series # e.g. burglary count for one borough
train, test = temporal_split(y)

baseline = seasonal_naive(train, season_length=12)
ets = ETSModel(train, error="add", trend="add", seasonal="add",
seasonal_periods=12).fit()
sarimax = SARIMAX(train, order=(1,1,1), seasonal_order=(0,1,1,12),
exog=train_exog).fit()

fc_baseline = baseline.forecast(len(test))
fc_ets = ets.forecast(len(test))
fc_sarimax = sarimax.forecast(len(test), exog=test_exog)
```

An ML / AI challenger sketch is reasonable only after the baseline is stable:

```
# Global ML challenger with lagged features
X, y = make_lag_features(panel_df, lags=[1,2,3,6,12], exog=["borough",
"crime_type", "holiday", "weather"])
model = XGBRegressor()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

If, later, LCIP pools many borough-by-category series with many covariates and you need multi-horizon forecasting, a TFT or DeepAR challenger becomes defensible. For the MVP vertical slice, it usually does not. ²²

Anomaly detection

For LCIP anomalies, the best first production choice is **decompose, then test**. STL gives an explicit trend/season/residual split; CUSUM and EWMA are classic and effective for detecting drift or small shifts. That makes the anomaly story easy to explain to a stakeholder: “we removed normal seasonality and flagged residual behaviour that exceeded a threshold or shift condition”. ²³

A statistical anomaly sketch:

```
stl = STL(monthly_count_series, period=12).fit()
resid = stl.resid
z = (resid - median(resid)) / mad(resid)
alerts = abs(z) > 3.5 # robust threshold
```

For multivariate anomalies, such as “borough-month tuples with unusual combinations of crime volume, month-on-month change, clearance patterns, and category mix”, Isolation Forest is a very good challenger because it is unsupervised and computationally light. Its core idea is that anomalies are easier to isolate and therefore have shorter path lengths in random partition trees. ²⁴

```
X = feature_frame[["count", "mom_change", "yoy_change", "rolling_std",
"clearance_rate"]]
clf = IsolationForest(contamination=0.02, random_state=42)
scores = clf.fit_predict(X)
anomalies = X[scores == -1]
```

For the MVP, I would keep deep anomaly detectors out of the critical path. They are harder to justify, explain, and validate in a domain where analysts will want to know exactly what “weird” means.

Classification

LCIP does not need person-level predictive policing to demonstrate machine-learning competence. A better MVP classification task is something bounded and governable, such as:

- classify whether an anomaly alert is likely to be “actionable” or “noise”;
- classify the likely type of user question for routing;
- classify whether a document chunk is policy, methodology, or operational context.

A regularised logistic regression is the right first model because it is transparent and probability-oriented. Scikit-learn also foregrounds calibration support, calibration curves, and Brier score evaluation, all of which are vital if output probabilities drive ranking or thresholding. ²⁵

```
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=200, class_weight="balanced")
)
pipe.fit(X_train, y_train)
proba = pipe.predict_proba(X_valid)[: , 1]
```

If you can show that the signal is genuinely nonlinear and that a simple model is leaving performance on the table, XGBoost is the next-best step. It is extremely strong for heterogeneous tabular data, and SHAP gives usable local explanations for interview and governance purposes. ²⁶

```
model = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8
)
model.fit(X_train, y_train)
shap_values = shap.TreeExplainer(model).shap_values(X_valid)
```

What I would **not** recommend for the MVP is any classifier intended to assess a specific person's offending or victimisation risk. That walks straight into the governance regime that Annex III treats as high-risk law-enforcement AI. ¹⁰

Retrieval and RAG

Retrieval is the clearest place where AI adds value in LCIP without taking over evidence generation. BM25 remains the right baseline because it is deterministic, exact-term friendly, cheap, and strong on domain-specific terms. Haystack's documentation is especially useful here: BM25 is lightweight, often used for exact matches such as names and IDs, and can outperform embedding approaches on out-of-domain queries. ⁸

Embeddings become useful because LCIP users will not always ask using the same exact vocabulary as the underlying data or notes. SBERT-style sentence embeddings are a mature and pragmatic choice, and compact models such as `all-MiniLM-L6-v2` map text into a 384-dimensional semantic space while remaining small enough for local deployment; Sentence Transformers lists that model at around 80 MB. ²⁷

The Lewis et al. RAG paper is still the canonical reason to use RAG in this setting: it combines parametric generation with explicit non-parametric memory, improving factual specificity while offering provenance and updateability. RAGAs then gives a sensible evaluation frame: retrieval relevance, faithfulness to context, and generation quality should be measured separately. ²⁸

A hybrid retrieval sketch:

```
# Pseudocode
docs = ingest_internal_docs()
bm25_index = build_bm25_index(docs)
vec_index = build_embedding_index(docs, model="all-MiniLM-L6-v2")

hits_sparse = bm25_index.search(query, filters=metadata_filters, k=20)
hits_dense = vec_index.search(query, filters=metadata_filters, k=20)
```

```
hits = reciprocal_rank_fusion(hits_sparse, hits_dense)[:8]
answer = grounded_llm(query, context=hits, require_citations=True)
```

For LCIP, retrieval should sit on top of structured filters. The right composition is usually: **metadata filter first, hybrid retrieval second, rerank third, generation last.**

Summarisation

Summarisation is where many MVPs become impressive demos but poor products. The right LCIP pattern is to split summarisation into two tiers.

Tier one is deterministic narrative generation. This powers the dashboard and always derives from trusted aggregates. It is reproducible, testable, and auditable.

```
def trend_summary(curr, prev, median_12m, borough, crime_type):
    change = (curr - prev) / max(prev, 1)
    return (
        f"{crime_type} in {borough} is {change:+.1%} month-on-month, "
        f"with {curr} incidents versus a 12-month median of {median_12m}."
    )
```

Tier two is analyst-facing grounded summarisation. This is where a local LLM earns its place: synthesising retrieved evidence, analyst notes, and structured facts into readable prose, but only with source citations and abstention when support is weak. NIST's Generative AI profile exists precisely because GenAI introduces risks beyond traditional software; NIST's draft snippets also stress measuring erroneous content and verifying citations in outputs. SummEval and later factuality work reinforce the same point: automatic metrics alone are not enough, and factual consistency matters. ²⁹

```
prompt = """
Use only the supplied evidence.
Cite each claim with the source id.
If evidence is insufficient, say 'Insufficient evidence'.
Return:
- summary
- bullet findings
- cited evidence
"""
reply = local_llm.generate(prompt, context=retrieved_chunks)
```

LCIP's local-first stack also makes small local models a reasonable operational choice for this assistant layer. Official model libraries list local options such as Llama 3.1 8B, Mistral 7B, and Phi-3 Mini 3.8B, which fit the MVP's "show AI locally" narrative without forcing a cloud dependency. ³⁰

Evaluation and validation

The interview-grade answer here is: **every model class gets a baseline, a regime-appropriate split strategy, and metrics that match how it will be used.**

For **forecasting**, evaluate on genuine out-of-sample periods, not residual fit. Forecasting: Principles and Practice is explicit that forecast accuracy must be judged on new data; TimeSeriesSplit and rolling-origin evaluation exist precisely because ordinary random CV leaks future information. Prophet's diagnostics documentation likewise recommends historical cutoffs for time-series cross-validation. In LCIP, this means rolling-origin validation by borough/category series, comparison against seasonal naïve, and significance checks on competing forecasts via Diebold–Mariano where appropriate. Good metrics are **MAE or MASE** for point accuracy and **prediction-interval coverage** for uncertainty quality. ³¹

For **anomaly detection**, labels are usually sparse. That shifts the evaluation emphasis from generic accuracy to operational usefulness: **precision at analyst review capacity**, **false alerts per month**, **mean lead time before a known event**, and **analyst adjudication agreement**. The anomaly-detection literature treats problem definition and context as central, which is why a labelled backtest set plus analyst review loop is more important than a single abstract score. ³²

For **classification**, use **average precision / PR-AUC** when the positive class is rare, and always evaluate **probability calibration** with reliability diagrams and **Brier score** when the output becomes a rank, threshold, or downstream risk signal. Accuracy alone is not enough. ³³

For **RAG**, do not collapse retrieval and answer generation into one opaque score. Evaluate at least three things separately: **retrieval relevance**, **faithfulness to retrieved context**, and **answer usefulness/quality**. RAGAs is explicitly built around that multi-dimensional view because RAG systems fail in different ways depending on whether the retriever found the right context or the generator used it properly. ³⁴

For **summarisation**, use humans for the final gate. ROUGE can help offline if you have reference summaries, but SummEval shows the limits of automatic metric selection and benchmark inconsistency. For LCIP, the right summarisation scorecard is usually: **factual correctness**, **completeness**, **citation fidelity**, **usefulness to analyst**, and **red-flag omission rate**. ³⁵

A concise validation policy for LCIP would be:

- promote a forecasting model only if it beats seasonal naïve on rolling-origin MASE and has sane interval coverage;
- promote an anomaly model only if it reduces analyst false-positive burden at a fixed recall target;
- promote a classifier only if PR-AUC improves **and** calibration remains acceptable;
- promote a RAG configuration only if retrieval relevance and answer faithfulness pass evaluation;
- never promote an LLM prompt/model change without human review on a fixed gold set. ³⁶

Governance, operationalisation, and deployment implications

The core governance principle is simple: **document the dataset**, **document the model**, **document the intended use**, and **log the full inference path**. Model Cards are designed precisely for communicating intended use, evaluation conditions, and performance characteristics, especially in high-impact domains. Datasheets for Datasets do the same for data provenance, composition, collection, and recommended uses. Together they provide the backbone for explainability, auditability, and interview-grade engineering maturity. ³⁷

On **hallucination and factual integrity**, LCIP should treat LLM output as a user-interface layer, not as a source of truth. The mitigations are straightforward and defensible: retrieve before generate, constrain prompts to provided evidence, require citations, surface supporting document snippets, and abstain

when support is insufficient. NIST's generative-AI guidance emphasises that generative systems bring distinct risks and that organisations should measure erroneous outputs and verify citations; RAG and RAGAs provide both the architectural and evaluative pattern for doing this responsibly. 38

On **bias and fairness**, the key LCIP move is to stay away from individual-level risk modelling in the MVP and to disaggregate performance across meaningful operational segments such as borough, offence type, season, and deprivation proxy where applicable. ICO guidance explicitly highlights fairness, transparency, and statistical accuracy, while NIST notes that accuracy reporting should use realistic test sets and may include disaggregated results by data segment. 39

On **drift and monitoring**, do not monitor only infrastructure. NVIDIA's production guidance is useful here because it separates data drift, model drift, prediction drift, model versioning, operational metrics, and cost monitoring. In practice LCIP should monitor: input schema and missingness, key feature distributions, forecast residual distributions, anomaly alert rates, classifier score distributions, retrieval hit composition, LLM abstention rate, citation failures, latency, and resource use. 40

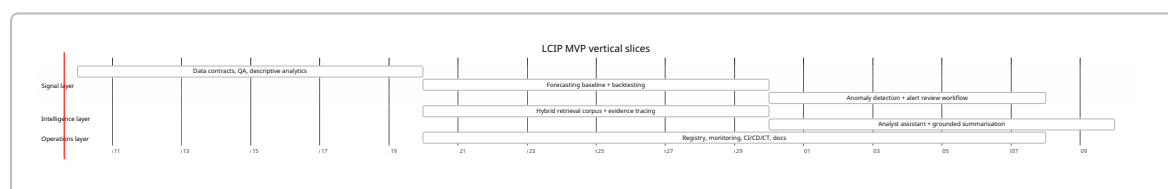
On **CI/CD and retraining**, Google's MLOps documentation is clear that ML systems need CI, CD, and CT rather than ad hoc notebook deployments. For LCIP that means: tests for feature generation, data-quality gates, reproducible training jobs, evaluation thresholds as release gates, and controlled promotion through staging aliases in a registry. MLflow gives the registry and lineage; OpenTelemetry gives cross-service observability; CI/CD/CT gives release discipline. 41

The deployment implications by method family are materially different:

- **Statistical models** are tiny, cheap, CPU-friendly, and easy to batch. They suit nightly or weekly retraining.
- **Logistic regression and XGBoost** remain lightweight enough for online use, with retraining typically monthly or when drift triggers.
- **Deep forecasting models** have higher training complexity, more dependencies, and a stronger justification burden.
- **Embeddings + hybrid retrieval** add an indexing pipeline, embedding versioning, and re-embedding overhead when the embedding model changes.
- **LLM summarisation** adds the highest latency and the largest failure surface, so in LCIP it belongs behind retrieval, with caching, guardrails, and human-reviewed gold sets. 42

Recommendations for the LCIP MVP

The highest-confidence recommendation is to build LCIP as **four vertical slices**, each demonstrating a different engineering competency while keeping the methodology proportionate.



This timeline is a design recommendation rather than a claim about your repository history; it reflects the research-backed ordering of simplest reliable value first, then increasingly sophisticated AI where it is justified. 43

What should use statistics first

Use statistics first for:

- **dashboard KPIs and trend analysis;**
- **forecasting monthly borough/offence counts;**
- **detection of unusual spikes, dips, and drift;**
- **template-generated executive summaries on the dashboard.** ⁴⁴

The reason is that these are the most rights-sensitive and evidence-bearing outputs in the system. They need to be reproducible, explainable, and defensible. This is where LCIP should shine as an analytics product rather than as a chatbot demo. ⁴⁵

What should use AI

Use AI for:

- **hybrid semantic retrieval over internal reports, policies, notebooks, and methodology;**
- **an analyst assistant that answers questions over retrieved evidence;**
- **grounded summarisation of retrieved context into readable briefings;**
- **optionally, challenger tabular models for classification or pooled forecasting once baselines and monitoring are in place.** ⁴⁶

This is where AI contributes genuine leverage: flexible access to knowledge, natural-language interaction, and synthesis over evidence, while the evidence itself remains anchored in deterministic or statistically interpretable pipelines. ²⁸

What to avoid in the MVP

Avoid, at least initially:

- person-level risk scoring, offender/victim prediction, or any output that looks like automated prioritisation of individuals;
- deep sequence models as the default forecasting solution;
- black-box anomaly models without a transparent baseline;
- free-form LLM summaries presented as authoritative facts without citations. ⁴⁷

Interview-friendly talking points

A hiring panel is likely to respond well if you can say the following, and mean it:

"I treated LCIP as a **high-trust analytics system**, so I separated the **signal layer** from the **language layer**. Core forecasts and anomalies come from interpretable statistical methods with proper backtesting. AI sits on top for retrieval and explanation, not underneath as an opaque source of operational truth." ⁴⁸

"I used the **simplest model that could plausibly win the job**, because Google's own ML engineering guidance warns that the real problems are often infrastructure, data, and monitoring rather than fancy modelling. I added challengers only after I had reliable baselines." ⁴⁹

"I chose methods by **failure mode**. If a failure could mislead an analyst about where crime is moving, I preferred deterministic or interpretable methods. If the failure mainly affected how evidence was presented, I allowed a grounded LLM with citations and abstention." 50

"I built the demo as if it might one day face scrutiny: model cards, dataset documentation, model registry lineage, observability, and explicit evaluation gates." 51

Open questions and limitations

A few recommendations would tighten further with project-specific information that is not fully established from the public repository and sources I reviewed: the exact number and granularity of LCIP time series to be forecast, the availability of labelled historical anomalies, the intended latency budget for the assistant, and whether any future roadmap includes individual-level policing decisions rather than area-level intelligence support. Those answers would mainly affect whether and when it becomes worthwhile to introduce pooled ML forecasters or more sophisticated anomaly challengers. 52

1 5 raw.githubusercontent.com

<https://raw.githubusercontent.com/Jay-Dee/lcip/main/README.md>

2 The M4 Competition: Results, findings, conclusion and way ...

https://ideas.repec.org/a/eee/intfor/v34y2018i4p802-808.html?utm_source=chatgpt.com

3 12 39 Guidance on AI and data protection | ICO

<https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/guidance-on-ai-and-data-protection/>

4 11 42 43 49 <https://developers.google.com/machine-learning/guides/rules-of-ml>

<https://developers.google.com/machine-learning/guides/rules-of-ml>

6 <https://arxiv.org/abs/1704.04110>

<https://arxiv.org/abs/1704.04110>

7 9 14 19 20 45 48 50 https://airc.nist.gov/AI_RMF_Knowledge_Base/AI_RMF/Foundational_Information/3-sec-characteristics

https://airc.nist.gov/AI_RMF_Knowledge_Base/AI_RMF/Foundational_Information/3-sec-characteristics

8 18 46 <https://docs.haystack.deepset.ai/docs/elasticsearchbm25retriever>

<https://docs.haystack.deepset.ai/docs/elasticsearchbm25retriever>

10 47 <https://ai-act-service-desk.ec.europa.eu/en/ai-act/annex-3>

<https://ai-act-service-desk.ec.europa.eu/en/ai-act/annex-3>

13 <https://mlflow.org/docs/latest/ml/model-registry/>

<https://mlflow.org/docs/latest/ml/model-registry/>

15 44 https://www.statsmodels.org/dev/generated/statsmodels.tsa.exponential_smoothing.ets.ETSModel.html

https://www.statsmodels.org/dev/generated/statsmodels.tsa.exponential_smoothing.ets.ETSModel.html

16 23 https://www.statsmodels.org/dev/examples/notebooks/generated/stl_decomposition.html

https://www.statsmodels.org/dev/examples/notebooks/generated/stl_decomposition.html

17 25 https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

- 21 https://facebook.github.io/prophet/static/prophet_paper_20170113.pdf
https://facebook.github.io/prophet/static/prophet_paper_20170113.pdf
- 22 26 <https://arxiv.org/abs/1603.02754>
<https://arxiv.org/abs/1603.02754>
- 24 <https://www.lamda.nju.edu.cn/publication/icdm08b.pdf>
<https://www.lamda.nju.edu.cn/publication/icdm08b.pdf>
- 27 <https://arxiv.org/abs/1908.10084>
<https://arxiv.org/abs/1908.10084>
- 28 <https://arxiv.org/abs/2005.11401>
<https://arxiv.org/abs/2005.11401>
- 29 38 <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- 30 <https://ollama.com/library/llama3.1%3A8b>
<https://ollama.com/library/llama3.1%3A8b>
- 31 36 <https://otexts.com/fpp3/accuracy.html>
<https://otexts.com/fpp3/accuracy.html>
- 32 <https://dl.acm.org/doi/10.1145/1541880.1541882>
<https://dl.acm.org/doi/10.1145/1541880.1541882>
- 33 https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html
https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html
- 34 <https://aclanthology.org/2024.eacl-demo.16/>
<https://aclanthology.org/2024.eacl-demo.16/>
- 35 <https://aclanthology.org/W04-1013.pdf>
<https://aclanthology.org/W04-1013.pdf>
- 37 51 <https://arxiv.org/abs/1810.03993>
<https://arxiv.org/abs/1810.03993>
- 40 <https://developer.nvidia.com/blog/a-guide-to-monitoring-machine-learning-models-in-production/>
<https://developer.nvidia.com/blog/a-guide-to-monitoring-machine-learning-models-in-production/>
- 41 <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
<https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- 52 <https://github.com/Jay-Dee/lcip>
<https://github.com/Jay-Dee/lcip>